

Application domains

Text processing: Ruby can be embedded into HTML, and has a clean and easy syntax that allows a new developer to learn it very quickly and easily.

CGI programming: Ruby can be used to write Common Gateway Interface (CGI) scripts. It can easily be connected to databases like DB2, MySQL, Oracle and Sybase.

Network programming: Network programming can be fun with Ruby's well-designed socket classes.

GUI programming: Ruby supports many GUI tools such as Tcl/Tk, GTK and OpenGL.

XML programming: Ruby has a rich set of built-in functions, which can be used directly into XML programming.

Prototyping: Ruby is often used to make prototypes that sometimes become production systems by replacing the bottlenecks with extensions written in C.

Programming fundamentals

1. Instructions and interpreters

Ruby is an 'interpreted' programming language, which means it can't run on your processor directly, but has to be fed into a middleman called the 'virtual machine' or VM. The VM takes in Ruby code on one side, and speaks natively to the operating system and the processor on the other. The benefit of this approach is that you can write Ruby code once and, typically, execute it on many different operating systems and hardware platforms. A Ruby program can't run on its own; you need to load the VM. There are two ways of executing Ruby with the VM — through IRB and through the command line.

Running Ruby from the command line: This is the durable way of writing Ruby code, because you save your instructions into a file. This file can then be backed up, transferred, added to source control, etc.

We might create a file named *my_program.rb* like this:

```
class Sample
  def hello
    puts "Hello World"
  end
end

s=Sample.new
s.hello
```

Then we could run the program in the terminal like this:

```
$ruby my_program.rb
Hello World
```

When you run *ruby my_program.rb* you're actually loading the Ruby virtual machine, which in turn loads your *my_program.rb*.

Running Ruby from IRB: IRB stands for 'interactive Ruby' and is a tool you can use to interactively execute Ruby expressions read from the standard input.

The *irb* command from your shell will start the interpreter.

2. Variables

Programming is all about creating abstractions, and in order to create an abstraction we must be able to assign names to things. Variables are a way of creating a name for a piece of data. In some languages you need to specify what type of data (like a number, word, etc) can go in a certain variable. Ruby, however, has a flexible type system where any variable can hold any kind of data.

Creating and assigning a variable: In some languages you need to 'declare' a variable before you assign a value to it. Ruby variables are automatically created when you assign a value to them. Let's try an example:

```
C:\Users\Administrator>irb
irb(main):001:0> a = 5
=> 5
irb(main):002:0> a
=> 5
```

The line *a = 5* creates the variable named 'a' and stores the value '5' into it.

Right side first: In English, we read left to right, so it's natural to read code left to right. But when evaluating an assignment using the single equals (=), Ruby actually evaluates the *right side first*. Take the following example:

```
C:\Users\Administrator>irb
irb(main):001:0> b = 10 + 5
b => 15
```

The *10 + 5* is evaluated first, and the result is given the name 'b'.

Flexible typing: Ruby's variables can hold any kind of data, and can even change the type of data they hold. For instance:

```
C:\Users\Administrator>irb
irb(main):001:0> c = 20
=> 20
irb(main):002:0> c = "hello"
=> "hello"
```

The first assignment gave the name 'c' to the number 20. The second assignment changed 'c' to the value 'hello'.

3. Strings

In the real world, strings tie things up. Programming strings have nothing to do with real-world strings. Programming strings are used to store collections of letters and numbers.